Spectroscopic Follow-up Orchestration Layer

Design & Implementation Overview for the MAGNETS Collaboration

Akum Gill
Harvard University
Collaborators: Chris Stubbs, Ashley Villar's group (Yize Dong), Conor Ransome

June 2026

Abstract

This document describes the design and implementation of the orchestration layer for automated spectroscopic follow-up of transients discovered by the RubinAlerts pipeline, targeting LLAMAS on Magellan/Baade for the MAGNETS collaboration. It covers alert ingestion, merit scoring, prioritization, scheduling, and multi-program time accounting, including a cross-night *per-target integration ledger* and *per-program light-curve phase preferences*. Two principles guide the design throughout: **no wasted time on sky** and **transparent, defensible prioritization** — an observer should be able to reconstruct why any target was ranked where it was. The system is exercised by 96 offline unit tests (no live broker/database calls).

Contents

1 Overview

The orchestrator converts ranked candidates into a LLAMAS observing plan with multi-program time accounting. It ingests targets from two streams:

1. **Alert-driven** — the RubinAlerts pipeline's `candidates.csv` (merit-ranked SN candidates aggregated across brokers); and
2. **Manual** — a CSV of PI-entered targets (the eventual Google-Sheet front end is future work; see §11).

The system is **LLAMAS-only** (Magellan/Baade; LDSS3 materials in `ref/` are reference only). The pipeline of modules is:

`normalize` → `prioritizer` → `planner` → `accounting` + `target_ledger` → `output/reporting`

2 Alert ingestion and liveness

2.1 Brokers

Candidates are aggregated from independent brokers, each with different classifiers, then cross-matched. ZTF data enters only via ALeRCE-ZTF and ANTARES; there is no standalone ZTF client.

Broker	Survey	Role
Fink	Rubin LSST	Primary LSST discovery + SN classification
ANTARES	ZTF + Rubin	Galaxy-centric cone search; P(Ia) is a heuristic *proxy* (not ML)
ALeRCE-ZTF	ZTF	BHRF forced-photometry classifier
ALeRCE-LSST	Rubin LSST	Stamp classifier (cone-search API ignores coordinates; queried globally then filtered locally)
ATLAS	—	Forced-photometry *enrichment* for bright (< 20 mag) candidates
TNS	—	Cross-match / duplicate & spectroscopic-confirmation check
Rubin RSP/TAP	Rubin LSST	Host-galaxy properties, authoritative photometry

2.2 Coverage and the Rubin downtime window

The southern Deep Drilling Fields (ELAIS-S1, EDFS_a/b, $\delta \lesssim -32°$) lie below ZTF/ANTARES/ALeRCE-DB coverage and are therefore **single-broker by construction**. This matters operationally: **Rubin LSST is offline 2026-08-09 through the end of August 2026**, during which the ZTF-fed brokers are the *primary* alert source and the southern DDFs lose discovery coverage entirely.

2.3 Liveness reporting

A silent broker outage must never be mistaken for a quiet sky — especially during the Rubin downtime, when the ZTF-fed brokers are the only source. The aggregator therefore records, per broker, a status record `{queried, responded, n_returned, error}`, written to a `broker_status.json`

sidecar and rendered as a "Broker Status" block in the run report. A broker that raises is recorded as queried-but-unresponsive rather than crashing the run, so the observer can always distinguish "no candidates" from "broker down".

2.4 Cross-matching

Cross-broker de-duplication uses true angular separation (`SkyCoord.separation`) against a fixed radius rather than an RA/Dec box test, whose effective radius would otherwise shrink with declination (worst in the southern fields).

3 Merit scoring (alert pipeline)

The alert pipeline ranks candidates by a *multiplicative* merit, so a candidate must score well on *every* axis:

$$\text{Merit} = w_{\text{time}}\, w_{\text{mag}}\, w_{\text{prob}}\, w_{\text{host}}\, w_{\text{ext}}\, w_{\text{broker}}\, w_{\text{moon}}\, (w_{\text{salt}}\, w_{\text{absmag}}). \tag{1}$$

- $w_{\text{time}} = \exp(-\Delta t^2/2\tau^2)$, $\tau = 10\,\text{d}$ — peaks at maximum light.
- $w_{\text{mag}} = \exp(-((m - m_{\text{opt}})/\sigma_m)^2)$, $m_{\text{opt}} \approx 20.5$ (Magellan S/N sweet spot).
- w_{prob} — **ML-derived P(Ia) only**. The ANTARES heuristic proxy is uncalibrated, so it is kept in a separate `antares_proxy_prob` column and excluded from the pooled probability; ANTARES contributes to detection/coverage, not to P(Ia).
- $w_{\text{host}}, w_{\text{ext}}$ — host-morphology and Galactic-extinction factors.
- w_{broker} — **coverage-aware**: $1 + 0.3\,\text{clip}\big(\frac{n-1}{n_{\max}-1}, 0, 1\big)$ where $n_{\max}$ is the number of brokers that *could* have seen the field given its declination. This ensures a single-broker southern field is not penalized relative to a single-broker equatorial field; full multi-broker coverage earns up to +30%.
- w_{moon} — folded into the ranking merit (not only into observability filtering), so targets close to a bright Moon are correctly down-weighted in the ranking itself. Merit is computed with the Moon penalty as a single source of truth, and `candidates.csv` (the orchestrator's input) carries the Moon-aware merit.

4 Prioritization (orchestrator)

4.1 Composite score

Each target receives a composite score that is a *multiplicative science core* plus two *additive nudges*:

$$\texttt{science\_term} = \texttt{SCIENCE\_SCALE} \times s \times b \times p \times c \tag{2}$$

$$\texttt{observability\_term} = \texttt{OBSERVABILITY\_BONUS} \times o \tag{3}$$

$$\texttt{keyword\_term} = \texttt{KEYWORD\_BONUS} \times k \tag{4}$$

$$\texttt{total} = \texttt{science\_term} + \texttt{observability\_term} + \texttt{keyword\_term} \tag{5}$$

with `SCIENCE_SCALE`= 100, `OBSERVABILITY_BONUS`= 20, `KEYWORD_BONUS`= 50. The scales are chosen so the science core ($s, b, p, c \in [0, 1]$) is the dominant axis (a P1 at peak on a healthy budget ≈ 100; a P4 ≈ 20), while observability and keyword nudges reorder within or across adjacent tiers but cannot by themselves leapfrog a full priority tier. The phase factor is clamped to $[0, 1]$ so it can never push the science core past its bound. Every target's component **breakdown is persisted** (`score_breakdown.json`) and printed in the summary, so a PI can reconstruct any ranking by hand — where:

s science	P1–P4 → 1.0/0.7/0.4/0.2
b budget	phase-aware factor 1.0/0.5/0.1 (§6)
p phase	per-program phase preference (§4.2)
c completeness	per-target integration ledger factor (§7)
o observability	fraction of night above the airmass limit
k keyword adj.	signed boosts/demotions from a controlled tag vocabulary, clamped

The keyword adjustment k is driven by a *controlled vocabulary* of scheduling tags (e.g. `urgent`, `too`, `near_peak`, `backup`, `low_priority`), not free-text parsing, so it cannot mis-fire on incidental words in an observer's notes; the net adjustment is clamped so keywords reorder within or across adjacent tiers but never leapfrog a full priority tier. A separate `override` (a.k.a. `mandatory`) tag is a hard, non-negotiable request: such a target bypasses normal ranking and the Moon-phase eligibility filter, and is *reserved* a slot before the greedy fill (see §5). Must-see authority is granted only to the night's designated *primary* program (§5); a non-primary program's override is downgraded to normal prioritization. It still cannot defy physics — a mandatory target that never clears the airmass limit tonight is reported as unschedulable rather than silently dropped.

4.2 Per-program light-curve phase preference

Motivation. Different programs want a SN at different phases. Cosmology / standardization science favors *peak* (best S/N, the canonical SALT epoch, and where luminosity-correlating spectral indicators are measured). Progenitor, circumstellar-material ("flash"), early-excess, and fast/exotic-transient science favor the *rise* (outer ejecta, narrow CSM lines that are gone by peak, and transients that evolve too fast to wait). A single peak-centered weight cannot serve both.

Implementation. The phase factor is computed relative to a per-program preferred epoch:

$$p = \exp\Big(-\frac{(\Delta t - \Delta t_{\rm pref})^2}{2\tau^2}\Big), \qquad \Delta t_{\rm pref} = \begin{cases} 0 & \text{preference } \texttt{peak} \\ -7\,\mathrm{d} & \text{preference } \texttt{rising} \end{cases} \tag{6}$$

with $\tau = 10\,\mathrm{d}$. Each `ProgramAllocation` carries a `phase_preference` (`peak` default; e.g. Stubbs/Ia = `peak`, Villar/exotic = `rising` in the example allocations). This requires a *signed* Δt (days from peak; < 0 rising), threaded from the `delta_t` column of `candidates.csv` or from `peak_mjd` for manual targets. The offset $\Delta t_{\rm pref} = -7\,\mathrm{d}$ is a tunable config value; its exact choice is a science-policy decision for the collaboration. Note an alert-sourced target without a `delta_t` value falls back to the legacy symmetric weight (peak-meaningful only), since the sign is unrecoverable from $w_{\rm time}$ alone.

4.3 Priority labels

Merit maps to P1–P4 by *tonight's quartiles*; these are **within-night relative** labels, not absolute science classes (a "P1" on a quiet night may be weaker than a "P4" on a rich one). The summary header states this caveat, and the < 4-target degenerate case is guarded by rank-based assignment. Absolute, cross-night-calibrated thresholds are a natural future refinement once merit statistics across many nights are available.

5 Scheduling

Each night has a designated *primary* program, looked up by date from a small nights CSV (`date, primary_program`). Only the primary's must-see (`mandatory`) targets are honored as guarantees;

any other program's override is logged and demoted to normal prioritization. Honored must-see targets are placed first in a reservation pass — each is given its best (lowest-airmass) slot within its observable window, and is kept even when that window is short — and the greedy fill then works around those reserved blocks. For the remaining targets, the greedy scheduler steps through the night, at each slot selecting the observable target maximizing

$$\text{score} - 10 \times \text{airmass} - \texttt{SLEW\_PENALTY} \times \theta_{\text{slew}}, \tag{7}$$

where θ_{slew} is the angular distance from the previously scheduled target (`SLEW_PENALTY`= 0.5). The slew term is a modest tie-breaker: the IFU's 1-minute overhead model assumes negligible slew, which is reasonable for clustered DDF targets, and this term guards against large cross-sky jumps between consecutive picks. The scoring mode (`prioritizer` vs. `fallback/priority-only`) is stamped into the summary header so it is always clear which ranking produced the plan. Standard stars are placed at start and end *and* interleaved at a configurable mid-night cadence (`standard_interleave_hours`= 3.5) for spectrophotometric calibration over a long night. Exposures are split into sub-exposures of at most `max_single_exposure_sec`= 900 s for cosmic-ray mitigation.

6 Time accounting

Per-program budgets are defined per moon phase (dark/grey/bright) in `allocations.yaml`. Time is **charged on schedule** and trued-up post-night by `reconcile`. Key properties:

- **Consistent state path.** `run-nightly` and `reconcile` derive the state file from the same `output_dir/time_accounting.json` convention (helper `_default_state_path`), so a reconciliation always updates the night it belongs to rather than charging against an empty ledger.
- **Science-time charging.** The budget is charged `charged_minutes` (exposure + overhead), not the padded wall-clock window; gap-fill / end-of-night stretch is tracked separately as `padding_minutes` for transparency but never billed.
- **Phase-aware budget factor.** The 1.0/0.5/0.1 budget factor uses remaining hours *in the relevant moon phase* as a fraction of that phase's allocation ($> 50\% \rightarrow 1.0$, $> 0 \rightarrow 0.5$, exhausted $\rightarrow 0.1$), so dark time is not throttled because grey time is short, and vice versa.

7 Per-target integration ledger

A cross-night ledger (`target_ledger.json`, stored beside the accounting state) answers a question the per-program budgets cannot: *does this target already have enough integration time that we should stop re-observing it?* Without it, five consecutive nights could keep re-scheduling the same high-merit object.

7.1 Identity, completeness, and remaining-time scheduling

Targets are keyed by sky coordinate (matched within $2''$ angular separation, so a name change between nights — e.g. an internal alert ID later upgraded to a TNS/IAU name — is still recognized as the same object; names are tracked as aliases with the TNS name preferred as canonical). For each target:

- `completeness_fraction` = cumulative science seconds / required seconds, where the required exposure is *recomputed every night* from the current magnitude/redshift — so a fading SN correctly needs more time.

- A target is **satisfied** (excluded from scheduling) at fraction ≥ 0.95 or when the remaining time is below a worthwhile 15-minute block.
- The completeness factor folded into the score is $\max(0.15, 1 - \text{fraction})$, dropping to 0 when fully done. The 0.15 floor prevents the opposite failure mode — a target getting one short exposure and then being dropped forever — while the taper prevents endless re-picking.
- A partially-observed target is scheduled for only its *remaining* needed time.

7.2 Phase-split integration count

The cumulative count is **split by phase bucket** derived from the observed Δt: `rising` ($\Delta t < -5$ d), `peak` ($|\Delta t| \leq 5$ d), `declining` ($\Delta t > 5$ d), or `all` when Δt is unknown (`phase_bucket_window_days`= 5). Completeness is judged against the bucket the SN is in *that night*, so a target that is "done at peak" can still be scheduled for needed rising-phase time, and vice versa. The ledger JSON format migrates old scalar records into the `all` bucket automatically.

7.3 Multi-group alert

When a single sky object is associated with more than one program — whether from two same-position requests on one night or from a program already recorded against that object on a prior night — the system emits an alert: a `logger.warning`, a "Multi-group Targets" section in the summary, and a `multi_group_alerts.json` sidecar with schema {`name, ra_deg, dec_deg, programs, phase_preferences, observed_phase, same_phase`}. The `same_phase` flag tells PIs whether the groups want the object at the same phase (one observation may serve both) or different phases (coordination / separate observations needed). Conflict *arbitration* is intentionally left to the PIs for now (see §11).

8 Exposure estimation

The orchestrator estimates LLAMAS exposure via a cascade: a redshift→time table from the proposal (**linearly interpolated** so there are no discontinuities between tabulated rows, and clamped outside the tabulated range), falling back to magnitude scaling (mag 20 $\approx$ 45 min, 2.5× per magnitude), then a 45-minute default. The IFU overhead is ∼1 min (vs. ∼10 min for slit instruments). Moon/airmass scaling is intentionally omitted here, since the proposal table is the reference for LLAMAS exposures; this is a deliberate divergence from the alert-pipeline estimator, not an inconsistency.

9 CLI and outputs

```
python -m orchestrator plan          --date ... --targets t.csv --moon
   grey
python -m orchestrator run-nightly   --date ... --candidates c.csv \
        --allocations a.yaml --moon grey --output-dir out/ \
        [--target-ledger out/target_ledger.json]
python -m orchestrator reconcile        --program P --actual-hours 3.5 ...
python -m orchestrator reconcile-target --name "SN␣2026abc" --actual-
   minutes 90 \
        [--phase peak] --date ...
python -m orchestrator ledger        --target-ledger out/target_ledger.
   json \
```

```
        [--show-satisfied]
```

Transparency artifacts written per run: the timeline, TCS catalog, and summary; plus `score_breakdown.json` (per-target score components), `time_accounting.json` (budget audit), `target_ledger.json` (per-phase integration), `broker_status.json` (ingestion liveness), and `multi_group_alerts.json`.

10 Testing

The system has 96 unit tests (`tests/`). A hard rule: **no test contacts a live broker, database, or API** — brokers are stubbed and all state uses temp-file JSON. Coverage spans merit (moon folding, coverage-aware broker bonus), ingestion (liveness status, ML-only probability, angular-separation dedup), accounting (state path, science-time charging, phase-aware budget factor), prioritization (normalized score, breakdown invariant, slew penalty, small-list guard), exposure interpolation, standards insertion, the integration ledger (completeness, phase splitting, migration), phase preferences, and multi-group alerts.

11 Open questions and future work

1. **Alert-target ownership / charging *(for collaboration discussion).*** Time is charged to each target's program, but alert-discovered candidates need an assigned owner. This depends on the operating model: **(a)** the tool is a shared scheduler that every group feeds its own targets into, and the alert stream simply serves as the Stubbs-group SN Ia source (alert candidates → the Ia program); or **(b)** multiple groups also draw targets from the alert stream, which would call for a shared alert budget or classification-based routing. The code currently assigns alert candidates to a single configurable default program; the policy is deferred to the collaboration.
2. **Manual-request front end.** A Google-Sheet ingester (gspread + service account) is planned so PIs can enqueue targets directly; the CSV path is the current stand-in. See §12 for the interface spec.
3. **Phase-preference tuning.** The `rising` offset (-7 d), τ, and the phase-bucket window (5 d) are science-policy values to be set with Chris and Ashley's group; per-program offsets (not just `peak`/`rising`) are a natural extension.
4. **Multi-group conflict arbitration.** Today we alert; a future option is to model an object wanted at two phases as two distinct requests, each with its own ledger bucket and required time.
5. **Absolute priority thresholds.** Replace within-night quartile P-labels with absolute merit thresholds once cross-night merit statistics are available.
6. **S/N-based exposure and completeness.** The exposure estimate is currently a static cascade that omits airmass, sky/Moon brightness, and seeing, and the ledger's completeness is measured in integrated *time*. The principled target is a proper exposure-time calculator (with airmass-scaled extinction and sky, Moon brightness, and seeing terms) and, ultimately, completeness judged from the *achieved* signal-to-noise rather than elapsed clock time. The ledger is already forward-compatible: cumulative time can be swapped for cumulative $\mathrm{S/N}^2$ (which adds linearly under background-limited conditions) with no change to its structure.
7. **Multi-night look-ahead** scheduling, and surfacing broker liveness through the `supernova_monitor` pipeline path (currently only the `run_tonight` path).

12 Planned: Google-Sheet manual-request front end

The intended interface for PI-entered targets (not yet implemented) reads a Google Sheet via `gspread` with a read-only service account; rows map onto the same `Target` schema the manual-CSV path already accepts (with the optional `program`, `phase_weight`/`peak_mjd`, and `delta_t` columns described above):

```
import gspread
from google.oauth2.service_account import Credentials

SCOPES = ['https://www.googleapis.com/auth/spreadsheets.readonly']

def fetch_sheet(sheet_id: str, range_name: str) -> list[dict]:
    creds = Credentials.from_service_account_file('credentials.json',
       scopes=SCOPES)
    client = gspread.authorize(creds)
    sheet = client.open_by_key(sheet_id)
    worksheet = sheet.worksheet(range_name)
    return worksheet.get_all_records()
```

An open design question is status feedback — how to mark targets "observed" back in the sheet once they are scheduled/satisfied.

13 References and proposal details

Proposal (Stubbs 2026B, propid 2835). Semester 2026B (Jul 7 – Jan 16); LLAMAS Integral Field Spectrograph on Magellan/Baade; 0.5D + 2.0G + 0.5B = 30 hours; 29 targets (SNe Ia in the DDFs + WD standards); $18.0 \leq r \leq 21.5$; $0.1 < z < 0.4$; queued observing.

Key papers. Adame et al. 2025 (DESI evolving dark-energy constraints); Boyd et al. 2026 (white-dwarf spectrophotometric standards); Ivezić et al. 2019 (LSST science drivers).

Local reference materials. Alex's LDSS3 ObsPlan generator and Yize's March observing-run notebook in `ref/` (reference only — the orchestrator is LLAMAS-only); the RubinAlerts pipeline (`run_tonight.py`, `core/magellan_planning.py`).